

Baobab-Studio AI 原生开发平台 详细设计文档

版本：v1.0
日期：2026-06-17
状态：待评审
关联文档：《JNPF 升维开发总计划 v5.0》第一篇 F-9 ~ F-10；第二篇 §3 Studio 确定版

一、文档概述

1.1 目标

本文档为 Baobab-Studio AI 原生开发平台的**前端菜单体系、权限模型、数据库结构、API 接口**提供施工级详细设计，覆盖以下范围：

- 四大一级菜单、31 个末端功能项的完整定义
- RBAC 权限模型（支持 A+C 先行，B 无缝扩展）
- 所有数据库表结构（DDL 级别）
- 所有 API 接口定义（路径、方法、参数、响应）
- 前端路由与组件映射
- 6 个 Sprint 的执行计划

1.2 读者

角色	关注章节
后端开发	三（权限模型）、四（数据库）、五（API）
前端开发	六（路由与组件）、三（权限前端实现）
产品经理	二（菜单全景）、七（Sprint 计划）
架构师	全文

1.3 术语

术语	含义
A 阶段	仅平台技术负责人可操作的功能（智能体配置等）
C 阶段	对租户有限开放的功能（行业知识、业务规则等）
B 阶段	后期扩展，租户可深度定制智能体（本文档不含实施，仅预留接口）
IR	Intermediate Representation, JNPF 中间表示
EAB	Enterprise Architecture Baseline, 企业架构基准
TenantId	租户标识，全链路隔离主键

二、菜单全景结构

2.1 一级菜单与 sort 顺序

一、AI 原生开发平台	sort=1	业务专家 / 开发者 / 租户管理员
二、智能体与流水线配置	sort=2	平台管理员（部分对租户开放）
三、JNPF 开发工具箱	sort=3	开发者
四、自博弈训练引擎	sort=4	创始人（TOTP 门禁）

2.2 完整菜单树

一、AI 原生开发平台	
├─ 1.1 提交需求	/studio/ai/submit-requirement
│ 用户描述想法 → 五阶段流水线逐步确认 → 生成系统	
│ 角色：全员	
├─ 1.2 已生成系统	/studio/ai/generated-systems
│ 系统列表 / 沙箱试用链接 / 源码下载 / 部署文档下载 / 再次提交改动	
│ 角色：全员	
├─ 1.3 UI 模板库	/studio/ai/ui-templates
│ Tab1 模板市场（业务专家选模板）	
│ Tab2 模板工坊（开发者创建/编辑模板）	
│ 角色：Tab1 全员 / Tab2 developer	
├─ 1.4 用量与计费	/studio/ai/usage-billing
│ Tab1 Token 用量统计	
│ Tab2 AI 调用明细	
│ Tab3 订阅续费	
│ 角色：平台管理员看全部 / 租户管理员看本租户 / 普通用户看自己	
二、智能体与流水线配置	
├─ 2.1 智能体管理	
│ ── 2.1.1 智能体创建与配置	/studio/agent/create
│ │ Prompt 加载 → 变量填充 → 子智能体关联 → Skills → MCP	
│ │ → 思维深度 → 测试运行	
│ │ 角色：platform_admin（A 阶段）	
│ ── 2.1.2 子智能体管理	/studio/agent/sub-agents
│ │ OrchestratorAgent 调度的子 Agent 配置	
│ │ 角色：platform_admin（A 阶段）	
│ ── 2.1.3 Skills 管理	/studio/agent/skills
│ │ 智能体可调用的工具/技能清单	
│ │ 角色：platform_admin（A 阶段）	
│ ── 2.1.4 MCP 配置	/studio/agent/mcp
│ │ Model Context Protocol 服务端连接	
│ │ 角色：platform_admin（A 阶段）	
├─ 2.2 流水线配置	
│ ── 2.2.1 流水线阶段设置	/studio/pipeline/stages
│ │ 五阶段各自的行为/文档模板/确认门槛/子 Agent 策略	

- | | 角色: platform_admin (A 阶段)
- | | | 2.2.2 模型路由策略 /studio/pipeline/model-routing
按阶段选模型 / 熔断阈值 / 重试次数
角色: platform_admin (A 阶段)
- | — 2.3 业务知识管理
 - | | — 2.3.1 业务规则配置中心 /studio/knowledge/rule-editor
决策表 / 决策树 / 规则链 / 来源标记 / 版本管理
角色: platform_admin:全部 / tenant_admin:本租户 (C 阶段)
 - | | | 2.3.2 领域知识管理 /studio/knowledge/domain-knowledge
领域模式 / 知识节点 / 图谱 / 统计 / 历史
角色: platform_admin:全部 / tenant_admin:只读 (C 阶段)
 - | | | 2.3.3 沙箱部署设置 /studio/knowledge/sandbox-config
CPU / 内存 / 超时 / 并发数 / 数据库策略
角色: platform_admin (A 阶段)
 - | | | 2.3.4 评测基准管理 /studio/knowledge/evals
golden set 管理 / 回归评测 / 分数趋势
角色: platform_admin (A 阶段)
- | — 2.4 租户定制 (仅 tenant_admin 可见)
 - | | — 2.4.1 行业知识设置 /studio/tenant/industry-knowledge
"我们是 XX 行业, 主要业务是..." 文本描述
角色: tenant_admin (C 阶段)
 - | | | 2.4.2 业务术语表 /studio/tenant/glossary
行业专业词汇 + 释义, 注入 AI 上下文
角色: tenant_admin (C 阶段)

三、JNPF 开发工具箱

- | — 3.1 领域模型画板 /studio/jnpf/domain-canvas
- | — 3.2 架构图设计器 /studio/jnpf/arch-designer
- | — 3.3 决策表编辑器 /studio/jnpf/decision-table
- | — 3.4 表单设计器 /studio/jnpf/form-designer
- | — 3.5 大屏设计器 /studio/jnpf/dashboard-designer
- | — 3.6 workflow 设计器 /studio/jnpf/workflow-designer
角色: developer

四、自博弈训练引擎

- | — 4.1 引擎总控
 - | | — 4.1.1 引擎开关与参数 /studio/foundry/engine-control
启停/暂停/总轮数/场景数/难度/超时
- | — 4.2 对抗角色配置
 - | | — 4.2.1 需求攻击者 /studio/foundry/agents/attacker
 - | | — 4.2.2 系统构建者 /studio/foundry/agents/builder
 - | | — 4.2.3 对抗性判官 /studio/foundry/agents/judge
 - | | — 4.2.4 知识蒸馏师 /studio/foundry/agents/distiller
- | — 4.3 训练运行
 - | | — 4.3.1 自博弈仪表盘 /studio/foundry/dashboard

└─ 4.3.2 因果回放池	/studio/foundry/causal-replay
└─ 4.3.3 沙箱集群管理	/studio/foundry/sandbox-cluster
└─ 4.4 领域知识进化	
└─ 4.4.1 领域模式	/studio/foundry/knowledge/patterns
└─ 4.4.2 知识节点	/studio/foundry/knowledge/nodes
└─ 4.4.3 知识图谱	/studio/foundry/knowledge/graph
└─ 4.4.4 使用统计	/studio/foundry/knowledge/stats
└─ 4.4.5 版本历史	/studio/foundry/knowledge/versions
└─ 4.4.6 反模式记录	/studio/foundry/knowledge/anti-patterns
└─ 4.4.7 叙事式说明	/studio/foundry/knowledge/narratives
└─ 4.4.8 冷启动种子	/studio/foundry/knowledge/cold-start
└─ 4.4.9 遗忘机制	/studio/foundry/knowledge/forgetting
└─ 4.5 知识补丁	
└─ 4.5.1 Patch 审核与签发	/studio/foundry/patch/review
└─ 4.5.2 Patch 接收日志	/studio/foundry/patch/logs
角色：founder（TOTP 门禁）	

三、权限模型

3.1 RBAC 角色定义

6 个角色，权限从高到低排列：

founder	创始人
额外要求：TOTP 二次认证	
范围：四、自博弈训练引擎全部	
二、智能体配置的系统级审计日志	
platform_admin	平台技术负责人
范围：二、智能体与流水线配置全部	
一、AI 原生开发平台的管理视图	
tenant_admin	租户管理员（客户方系统管理员）
范围：一、AI 原生开发平台全部	
二.4 租户定制（行业知识+术语表）	
二.3.1 业务规则配置中心（本租户）	
二.3.2 领域知识管理（只读）	
developer	开发者（客户方技术人员）
范围：一、AI 原生开发平台全部	
三、JNPF 开发工具箱全部	
business_expert	业务专家（客户方业务人员）
范围：一、AI 原生开发平台（不含模板工坊）	
normal_user	普通用户
范围：一、AI 原生开发平台基础功能	
用量与计费（仅看自己）	

3.2 一个用户可以同时拥有多个角色

```
-- 用户-角色关联表
-- 张三可以同时是 tenant_admin + developer
CREATE TABLE BASE_USER_ROLE (
    F_Id          BIGINT PRIMARY KEY,          -- 雪花主键
    F_UserId      BIGINT NOT NULL,             -- 用户 ID
    F_RoleCode    NVARCHAR(50) NOT NULL,       -- 角色编码
    F_TenantId    NVARCHAR(50) NOT NULL,       -- 租户 ID
    F_CreatorTime DATETIME NOT NULL,
    F_CreatorUserId BIGINT,
    CONSTRAINT UQ_USER_ROLE UNIQUE (F_UserId, F_RoleCode, F_TenantId)
);
```

3.3 菜单权限控制模型

核心思路：一个菜单项 = 一条权限记录。每条记录定义了"哪些角色可以访问"和"访问级别是什么"。

菜单权限 = 角色 × 数据范围

角色决定：能不能看到这个菜单

数据范围决定：看到的数据是全部、本租户、还是只有自己的

数据范围等级：

级别	含义	示例
ALL	看所有人数据	平台管理员看所有租户的 AI 调用记录
TENANT	看本租户数据	租户管理员看自己企业的规则
OWN	只看自己的数据	普通用户看自己的 Token 用量
NONE	不可见	普通用户看不到智能体配置菜单

3.4 扩展 B 的预留设计

每个菜单项有一个 F_ExpandPhase 字段：

'A' → A 阶段已实现，仅平台管理员
'C' → C 阶段已实现，对租户有限开放
'B' → 预留，后期开放给租户深度定制

扩展到 B 时，只需要：

1. 把菜单记录的 F_RequiredRoles 里加上 tenant_admin
2. 如果需要简化视图，配一个 F_TenantViewConfig (JSON，定义租户版看到哪些字段)
3. 前端根据角色自动渲染对应视图

不需要新建表、不需要改 API、不需要改权限拦截逻辑。

四、数据库设计

4.1 菜单表 BASE_MENU

```
CREATE TABLE BASE_MENU (  
    F_Id                BIGINT PRIMARY KEY,                -- 雪花主键  
    F_ParentId          BIGINT DEFAULT 0,                  -- 父菜单ID, 0=顶级  
    F_Name              NVARCHAR(100) NOT NULL,            -- 菜单名称  
    F_Icon              NVARCHAR(100),                     -- 菜单图标  
    F_Url               NVARCHAR(500),                     -- 前端路由路径  
    F_Sort              INT NOT NULL DEFAULT 0,            -- 排序号  
    F_Enabled           BIT DEFAULT 1,                     -- 是否启用  
    F_IsVisible         BIT DEFAULT 1,                     -- 是否可见 (默认显示)  
    F_IsPublic          BIT DEFAULT 0,                     -- 是否全员可见  
    F_Comment           NVARCHAR(500),                     -- 备注说明  
  
    -- ★ 权限控制字段  
    F_RequiredRoles     NVARCHAR(500),                     -- 允许的角色列表(JSON数组)  
    F_DataScope         NVARCHAR(20) DEFAULT 'NONE',       -- 数据范围:  
    ALL/TENANT/OWN/NONE  
    F_ExpandPhase       CHAR(1) DEFAULT 'A',              -- 扩展阶段: A/B/C  
  
    -- ★ B 阶段扩展预留  
    F_TenantViewConfig  NVARCHAR(MAX),                    -- 租户版简化视图配置(JSON)  
  
    -- 审计字段  
    F_CreatorTime       DATETIME NOT NULL,  
    F_CreatorUserId     BIGINT,  
    F_CreatorUserName   NVARCHAR(50),  
    F_ModifyTime        DATETIME,  
    F_ModifyUserId      BIGINT,  
    F_ModifyUserName    NVARCHAR(50),  
    F_DeleteMark        BIT DEFAULT 0  
);  
  
-- 索引  
CREATE INDEX IX_MENU_PARENT ON BASE_MENU(F_ParentId, F_Sort);  
CREATE INDEX IX_MENU_URL ON BASE_MENU(F_Url);
```

4.2 菜单初始数据

```
-- ===== 一、AI 原生开发平台 =====  
INSERT INTO BASE_MENU (F_Id, F_ParentId, F_Name, F_Sort, F_Url, F_RequiredRoles,  
F_DataScope, F_ExpandPhase, F_IsPublic, F_Comment)  
VALUES  
-- 一级菜单  
(100000001, 0, N'AI 原生开发平台', 1, NULL, N'[]', 'NONE', 'A', 1, N'面向业务专家/开发  
者的主功能区'),  
  
-- 1.1 提交需求  
(100000101, 100000001, N'提交需求', 1, N'/studio/ai/submit-requirement',  
  
N'["platform_admin","founder","tenant_admin","developer","business_expert","normal_us  
er"]',  
    'NONE', 'A', 1, N'五阶段流水线入口, 全员可用'),
```

```

-- 1.2 已生成系统
(100000102, 100000001, N'已生成系统', 2, N'/studio/ai/generated-systems',

N'["platform_admin","founder","tenant_admin","developer","business_expert","normal_us
er"]',
'TENANT', 'A', 1, N'用户只能看到自己租户的系统，带红点数字提示'),

-- 1.3 UI 模板库
(100000103, 100000001, N'UI 模板库', 3, N'/studio/ai/ui-templates',

N'["platform_admin","founder","tenant_admin","developer","business_expert","normal_us
er"]',
'NONE', 'A', 1, N'Tab1模板市场全员/Tab2模板工坊仅developer'),

-- 1.4 用量与计费
(100000104, 100000001, N'用量与计费', 4, N'/studio/ai/usage-billing',

N'["platform_admin","founder","tenant_admin","developer","business_expert","normal_us
er"]',
'OWN', 'A', 1, N'平台管理员看ALL，租户管理员看TENANT，其他人看OWN');

-- ===== 二、智能体与流水线配置 =====
INSERT INTO BASE_MENU (F_Id, F_ParentId, F_Name, F_Sort, F_Url, F_RequiredRoles,
F_DataScope, F_ExpandPhase)
VALUES
-- 一级菜单
(200000001, 0, N'智能体与流水线配置', 2, NULL, N'[]', 'NONE', 'A'),

-- 2.1 智能体管理（二级菜单容器）
(200000100, 200000001, N'智能体管理', 1, NULL, N'[]', 'NONE', 'A'),

-- 2.1.1 智能体创建与配置
(200000101, 200000100, N'智能体创建与配置', 1, N'/studio/agent/create',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.1.2 子智能体管理
(200000102, 200000100, N'子智能体管理', 2, N'/studio/agent/sub-agents',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.1.3 skills 管理
(200000103, 200000100, N'skills 管理', 3, N'/studio/agent/skills',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.1.4 MCP 配置
(200000104, 200000100, N'MCP 配置', 4, N'/studio/agent/mcp',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.2 流水线配置（二级菜单容器）
(200000200, 200000001, N'流水线配置', 2, NULL, N'[]', 'NONE', 'A'),

-- 2.2.1 流水线阶段设置
(200000201, 200000200, N'流水线阶段设置', 1, N'/studio/pipeline/stages',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.2.2 模型路由策略
(200000202, 200000200, N'模型路由策略', 2, N'/studio/pipeline/model-routing',
N'["platform_admin"]', 'ALL', 'A'),

```

```

-- 2.3 业务知识管理（二级菜单容器）
(200000300, 200000001, N'业务知识管理', 3, NULL, N'[]', 'NONE', 'A'),

-- 2.3.1 业务规则配置中心
(200000301, 200000300, N'业务规则配置中心', 1, N'/studio/knowledge/rule-editor',
N'["platform_admin","tenant_admin"]', 'TENANT', 'C'),

-- 2.3.2 领域知识管理
(200000302, 200000300, N'领域知识管理', 2, N'/studio/knowledge/domain-knowledge',
N'["platform_admin","tenant_admin"]', 'TENANT', 'C'),

-- 2.3.3 沙箱部署设置
(200000303, 200000300, N'沙箱部署设置', 3, N'/studio/knowledge/sandbox-config',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.3.4 评测基准管理
(200000304, 200000300, N'评测基准管理', 4, N'/studio/knowledge/evals',
N'["platform_admin"]', 'ALL', 'A'),

-- 2.4 租户定制（二级菜单容器）
(200000400, 200000001, N'租户定制', 4, NULL,
N'["tenant_admin"]', 'NONE', 'C'),

-- 2.4.1 行业知识设置
(200000401, 200000400, N'行业知识设置', 1, N'/studio/tenant/industry-knowledge',
N'["tenant_admin"]', 'TENANT', 'C'),

-- 2.4.2 业务术语表
(200000402, 200000400, N'业务术语表', 2, N'/studio/tenant/glossary',
N'["tenant_admin"]', 'TENANT', 'C');

-- ===== 三、JNPF 开发工具箱 =====
INSERT INTO BASE_MENU (F_Id, F_ParentId, F_Name, F_Sort, F_Ur[, F_RequiredRoles,
F_DataScope, F_ExpandPhase)
VALUES
(300000001, 0, N'JNPF 开发工具箱', 3, NULL, N'[]', 'NONE', 'A'),

(300000101, 300000001, N'领域模型画板', 1, N'/studio/jnpf/domain-canvas',
N'["developer"]', 'TENANT', 'A'),
(300000102, 300000001, N'架构图设计器', 2, N'/studio/jnpf/arch-designer',
N'["developer"]', 'TENANT', 'A'),
(300000103, 300000001, N'决策表编辑器', 3, N'/studio/jnpf/decision-table',
N'["developer"]', 'TENANT', 'A'),
(300000104, 300000001, N'表单设计器', 4, N'/studio/jnpf/form-designer',
N'["developer"]', 'TENANT', 'A'),
(300000105, 300000001, N'大屏设计器', 5, N'/studio/jnpf/dashboard-designer',
N'["developer"]', 'TENANT', 'A'),
(300000106, 300000001, N' workflow 设计器', 6, N'/studio/jnpf/workflow-designer',
N'["developer"]', 'TENANT', 'A');

-- ===== 四、自博弈训练引擎 =====
INSERT INTO BASE_MENU (F_Id, F_ParentId, F_Name, F_Sort, F_Ur[, F_RequiredRoles,
F_DataScope, F_ExpandPhase)
VALUES
(400000001, 0, N'自博弈训练引擎', 4, NULL, N'["founder"]', 'ALL', 'A'),

```

-- 4.1 引擎总控

```
(400000100, 400000001, N'引擎总控', 1, NULL, N'["founder"]', 'ALL', 'A'),  
(400000101, 400000100, N'引擎开关与参数', 1, N'/studio/foundry/engine-control',  
N'["founder"]', 'ALL', 'A'),
```

-- 4.2 对抗角色配置

```
(400000200, 400000001, N'对抗角色配置', 2, NULL, N'["founder"]', 'ALL', 'A'),  
(400000201, 400000200, N'需求攻击者', 1, N'/studio/foundry/agents/attacker',  
N'["founder"]', 'ALL', 'A'),  
(400000202, 400000200, N'系统构建者', 2, N'/studio/foundry/agents/builder',  
N'["founder"]', 'ALL', 'A'),  
(400000203, 400000200, N'对抗性判官', 3, N'/studio/foundry/agents/judge',  
N'["founder"]', 'ALL', 'A'),  
(400000204, 400000200, N'知识蒸馏师', 4, N'/studio/foundry/agents/distiller',  
N'["founder"]', 'ALL', 'A'),
```

-- 4.3 训练运行

```
(400000300, 400000001, N'训练运行', 3, NULL, N'["founder"]', 'ALL', 'A'),  
(400000301, 400000300, N'自博弈仪表盘', 1, N'/studio/foundry/dashboard',  
N'["founder"]', 'ALL', 'A'),  
(400000302, 400000300, N'因果回放池', 2, N'/studio/foundry/causal-replay',  
N'["founder"]', 'ALL', 'A'),  
(400000303, 400000300, N'沙箱集群管理', 3, N'/studio/foundry/sandbox-cluster',  
N'["founder"]', 'ALL', 'A'),
```

-- 4.4 领域知识进化

```
(400000400, 400000001, N'领域知识进化', 4, NULL, N'["founder"]', 'ALL', 'A'),  
(400000401, 400000400, N'领域模式', 1, N'/studio/foundry/knowledge/patterns',  
N'["founder"]', 'ALL', 'A'),  
(400000402, 400000400, N'知识节点', 2, N'/studio/foundry/knowledge/nodes',  
N'["founder"]', 'ALL', 'A'),  
(400000403, 400000400, N'知识图谱', 3, N'/studio/foundry/knowledge/graph',  
N'["founder"]', 'ALL', 'A'),  
(400000404, 400000400, N'使用统计', 4, N'/studio/foundry/knowledge/stats',  
N'["founder"]', 'ALL', 'A'),  
(400000405, 400000400, N'版本历史', 5, N'/studio/foundry/knowledge/versions',  
N'["founder"]', 'ALL', 'A'),  
(400000406, 400000400, N'反模式记录', 6, N'/studio/foundry/knowledge/anti-patterns',  
N'["founder"]', 'ALL', 'A'),  
(400000407, 400000400, N'叙事式说明', 7, N'/studio/foundry/knowledge/narratives',  
N'["founder"]', 'ALL', 'A'),  
(400000408, 400000400, N'冷启动种子', 8, N'/studio/foundry/knowledge/cold-start',  
N'["founder"]', 'ALL', 'A'),  
(400000409, 400000400, N'遗忘机制', 9, N'/studio/foundry/knowledge/forgetting',  
N'["founder"]', 'ALL', 'A'),
```

-- 4.5 知识补丁

```
(400000500, 400000001, N'知识补丁', 5, NULL, N'["founder"]', 'ALL', 'A'),  
(400000501, 400000500, N'Patch 审核与签发', 1, N'/studio/foundry/patch/review',  
N'["founder"]', 'ALL', 'A'),  
(400000502, 400000500, N'Patch 接收日志', 2, N'/studio/foundry/patch/logs',  
N'["founder"]', 'ALL', 'A');
```

4.3 行业知识设置表 (C 阶段新增)

-- 租户行业知识描述

```
CREATE TABLE BASE_TENANT_INDUSTRY (
  F_Id BIGINT PRIMARY KEY,
  F_TenantId NVARCHAR(50) NOT NULL, -- 租户ID
  F_IndustryName NVARCHAR(200) NOT NULL, -- 行业名称
  F_Description NVARCHAR(MAX), -- 行业描述
  F_KeyScenarios NVARCHAR(MAX), -- 关键业务场景(JSON数组)
  F_SystemPrompt NVARCHAR(MAX), -- 注入AI的行业上下文(自动生成)
  F_Enabled BIT DEFAULT 1,
  F_CreatorTime DATETIME NOT NULL,
  F_CreatorUserId BIGINT,
  F_ModifyTime DATETIME,
  F_ModifyUserId BIGINT,
  F_DeleteMark BIT DEFAULT 0,
  CONSTRAINT UQ_TENANT_INDUSTRY UNIQUE (F_TenantId)
);
```

4.4 业务术语表 (C 阶段新增)

-- 租户业务术语表

```
CREATE TABLE BASE_TENANT_GLOSSARY (
  F_Id BIGINT PRIMARY KEY,
  F_TenantId NVARCHAR(50) NOT NULL, -- 租户ID
  F_Term NVARCHAR(200) NOT NULL, -- 术语
  F_Definition NVARCHAR(2000) NOT NULL, -- 定义/释义
  F_Synonyms NVARCHAR(500), -- 同义词(JSON数组)
  F_Category NVARCHAR(100), -- 分类(如: 业务/技术/组织)
  F_Example NVARCHAR(1000), -- 使用示例
  F_Enabled BIT DEFAULT 1,
  F_CreatorTime DATETIME NOT NULL,
  F_CreatorUserId BIGINT,
  F_ModifyTime DATETIME,
  F_ModifyUserId BIGINT,
  F_DeleteMark BIT DEFAULT 0,
  CONSTRAINT UQ_TENANT_TERM UNIQUE (F_TenantId, F_Term)
);
```

```
CREATE INDEX IX_GLOSSARY_TENANT ON BASE_TENANT_GLOSSARY(F_TenantId, F_Category);
```

4.5 已生成系统表 (1.2 菜单数据源)

-- 用户通过流水线生成的系统记录

```
CREATE TABLE BASE_AI_GENERATED_PROJECT (
  F_Id BIGINT PRIMARY KEY,
  F_TenantId NVARCHAR(50) NOT NULL,
  F_UserId BIGINT NOT NULL, -- 提交需求的用户
  F_ProjectName NVARCHAR(200) NOT NULL, -- 系统名称
  F_Description NVARCHAR(MAX), -- 需求描述

  -- 流水线状态
  F_PipelineStatus NVARCHAR(20) NOT NULL DEFAULT 'stage1',
  -- stage1/stage2/stage3/stage4/stage5/completed/failed

  F_CurrentStage INT DEFAULT 1, -- 当前阶段(1-5)
```

```

-- 产出物
F_SandboxUrl NVARCHAR(500), -- 沙箱试用链接
F_SandboxAccount NVARCHAR(100) DEFAULT 'admin', -- 试用账号
F_SandboxPassword NVARCHAR(100) DEFAULT '123456', -- 试用密码
F_SourceZipUrl NVARCHAR(500), -- 源代码下载链接
F_DeployDocUrl NVARCHAR(500), -- 部署说明文档链接

-- IR 存储
F_RequirementIR NVARCHAR(MAX), -- 需求分析IR(JSON)
F_ArchitectureIR NVARCHAR(MAX), -- 架构设计IR(JSON)
F_DesignIR NVARCHAR(MAX), -- 详细设计IR(JSON)
F_FinalIR NVARCHAR(MAX), -- 最终完整IR(JSON)

-- 红点提示
F_IsRead BIT DEFAULT 0, -- 用户是否已读
F_UpdateCount INT DEFAULT 0, -- 未读更新次数

-- 审计
F_CreatorTime DATETIME NOT NULL,
F_CreatorUserId BIGINT,
F_ModifyTime DATETIME,
F_ModifyUserId BIGINT,
F_DeleteMark BIT DEFAULT 0
);

CREATE INDEX IX_PROJECT_TENANT ON BASE_AI_GENERATED_PROJECT(F_TenantId, F_UserId);
CREATE INDEX IX_PROJECT_STATUS ON BASE_AI_GENERATED_PROJECT(F_PipelineStatus);
CREATE INDEX IX_PROJECT_UNREAD ON BASE_AI_GENERATED_PROJECT(F_TenantId, F_IsRead)
WHERE F_DeleteMark = 0;

```

4.6 Token 用量记录表 (1.4 数据源)

```

-- 复用文档中已定义的 BASE_AI_CALL_LOG，扩展字段
CREATE TABLE BASE_AI_CALL_LOG (
    F_Id BIGINT PRIMARY KEY,
    F_TenantId NVARCHAR(50) NOT NULL,
    F_UserId BIGINT NOT NULL,
    F_UserName NVARCHAR(50),

    -- 调用信息
    F_Provider NVARCHAR(50) NOT NULL, --
    F_Model NVARCHAR(100) NOT NULL, -- 模型名称
    F_Stage INT, -- 流水线阶段(1-5)
    F_AgentType NVARCHAR(50), -- 智能体类型

    -- 用量
    F_PromptTokens INT NOT NULL DEFAULT 0,
    F_CompletionTokens INT NOT NULL DEFAULT 0,
    F_TotalTokens INT NOT NULL DEFAULT 0,
    F_Latency INT NOT NULL DEFAULT 0, -- 延迟(ms)

    -- 结果
    F_Status NVARCHAR(20) NOT NULL, -- success/failed/timeout
    F_ErrorMessage NVARCHAR(2000),

    -- 成本估算
    F_EstimatedCost DECIMAL(10,6), -- 估算费用(元)

```

```

        F_CreatorTime    DATETIME NOT NULL
    );

CREATE INDEX IX_CALLLOG_TENANT_USER ON BASE_AI_CALL_LOG(F_TenantId, F_UserId,
F_CreatorTime);
CREATE INDEX IX_CALLLOG_PROVIDER ON BASE_AI_CALL_LOG(F_Provider, F_CreatorTime);

```

4.7 红点提示表 (1.2 菜单冒红点数据源)

```

-- 菜单红点提示（通用机制，不仅限于"已生成系统"）
CREATE TABLE BASE_MENU_BADGE (
    F_Id                BIGINT PRIMARY KEY,
    F_MenuId            BIGINT NOT NULL,           -- 菜单ID
    F_UserId            BIGINT NOT NULL,          -- 用户ID
    F_TenantId          NVARCHAR(50) NOT NULL,
    F_Count             INT DEFAULT 0,           -- 未读数量
    F_ExtraData         NVARCHAR(MAX),           -- 附加数据(JSON，如具体系统ID列
表)
    F_CreatorTime       DATETIME NOT NULL,
    F_ModifyTime        DATETIME,
    CONSTRAINT UQ_BADGE UNIQUE (F_MenuId, F_UserId, F_TenantId)
);

```

4.8 AI 会话与流水线表

```

-- AI 会话主表（提交需求时创建一个会话）
CREATE TABLE BASE_AI_PIPELINE (
    F_Id                BIGINT PRIMARY KEY,
    F_TenantId          NVARCHAR(50) NOT NULL,
    F_UserId            BIGINT NOT NULL,
    F_ProjectId         BIGINT,                  -- 关联已生成系统ID
    F_CurrentStage      INT DEFAULT 1,
    F_Status            NVARCHAR(20) DEFAULT 'active', -- active/completed/failed
    F_CreatorTime       DATETIME NOT NULL,
    F_ModifyTime        DATETIME,
    F_DeleteMark        BIT DEFAULT 0
);

-- AI 会话消息表（每轮对话记录）
CREATE TABLE BASE_AI_PIPELINE_MESSAGE (
    F_Id                BIGINT PRIMARY KEY,
    F_PipelineId        BIGINT NOT NULL,         -- 关联会话ID
    F_TenantId          NVARCHAR(50) NOT NULL,
    F_Stage             INT NOT NULL,            -- 第几阶段
    F_Role              NVARCHAR(20) NOT NULL,   -- user/assistant/system
    F_Content           NVARCHAR(MAX) NOT NULL,  -- 消息内容
    F_ContentType       NVARCHAR(20) DEFAULT 'text', -- text/json/ir/document
    F_AttachmentUrl     NVARCHAR(500),          -- 附件URL
    F_IsConfirmed       BIT DEFAULT 0,          -- 用户是否确认
    F_CreatorTime       DATETIME NOT NULL
);

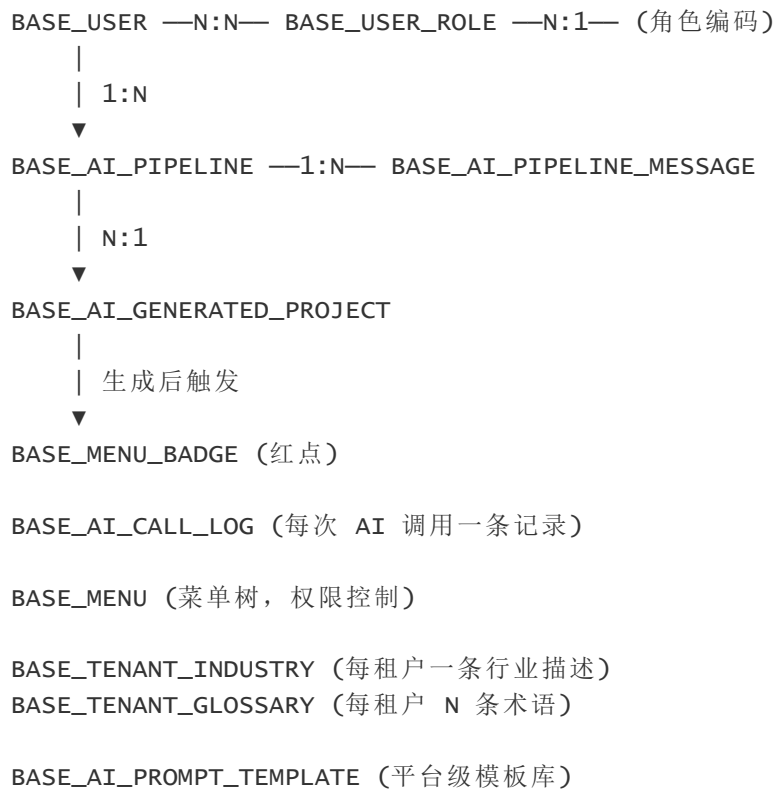
CREATE INDEX IX_PIPELINE_MSG ON BASE_AI_PIPELINE_MESSAGE(F_PipelineId, F_Stage,
F_CreatorTime);

```

4.9 Prompt 模板表

```
CREATE TABLE BASE_AI_PROMPT_TEMPLATE (  
    F_Id BIGINT PRIMARY KEY,  
    F_TemplateCode NVARCHAR(100) NOT NULL UNIQUE, -- 模板编码  
    F_Name NVARCHAR(200) NOT NULL, -- 模板名称  
    F_SystemPrompt NVARCHAR(MAX) NOT NULL, -- System Prompt 模板  
    F_Variables NVARCHAR(MAX), -- 变量定义(JSON数组)  
    F_AgentType NVARCHAR(50), -- 关联智能体类型  
    F_Version INT DEFAULT 1,  
    F_Enabled BIT DEFAULT 1,  
    F_CreatorTime DATETIME NOT NULL,  
    F_CreatorUserId BIGINT,  
    F_ModifyTime DATETIME,  
    F_ModifyUserId BIGINT,  
    F_DeleteMark BIT DEFAULT 0  
);
```

4.10 表关系总览



五、API 接口设计

所有接口遵循 JNPF DynamicApi 规范，Service 实现 IDynamicApiController。

5.1 菜单与权限接口

GET /api/menu/user-menus

说明：获取当前用户可见的菜单树

响应：根据用户角色过滤后的完整菜单树(含红点)

POST /api/menu/badge/read

说明：标记菜单已读（清除红点）

参数：{ menuId: number, projectId?: number }

5.2 AI 原生开发平台接口

-- 1.1 提交需求

POST /api/ai/pipeline/create

说明：创建 AI 会话，开始五阶段流水线

参数：{ requirement: string, attachments?: File[] }

响应：{ pipelineId, currentStage }

POST /api/ai/pipeline/{id}/message

说明：向流水线发送消息（回答 AI 的追问 / 补充需求）

参数：{ content: string, attachments?: File[] }

响应：SSE 流式

POST /api/ai/pipeline/{id}/confirm

说明：确认当前阶段产出，推进到下一阶段

参数：{ stage: number, approved: boolean, feedback?: string }

GET /api/ai/pipeline/{id}/state

说明：获取流水线当前状态

响应：{ currentStage, status, messages[], stageOutputs{} }

-- 1.2 已生成系统

GET /api/ai/project/list

说明：获取已生成系统列表（按租户过滤）

响应：{ items: [{ id, name, status, sandboxUrl, sourceZipUrl, deployDocUrl, isRead, updateCount }] }

POST /api/ai/project/{id}/mark-read

说明：标记已读

-- 1.3 UI 模板库

GET /api/ai/ui-template/market

说明：获取模板市场列表

响应：{ items: [{ id, name, thumbnail, category, designer }] }

GET /api/ai/ui-template/workshop

说明：获取开发者创建的模板（仅 developer）

POST /api/ai/ui-template/create

说明：创建 UI 模板（仅 developer）

-- 1.4 用量与计费

GET /api/ai/usage/summary

说明：Token 用量汇总（根据角色决定数据范围）

参数：{ startDate, endDate }

响应：{ totalTokens, totalCost, byProvider[], byStage[] }

```
GET /api/ai/usage/call-log
说明: AI 调用明细列表
参数: { page, pageSize, provider?, stage?, startDate?, endDate? }

GET /api/ai/usage/billing
说明: 订阅续费信息
响应: { plan, expiredAt, autoRenew, price }
```

5.3 智能体配置接口 (仅 platform_admin)

```
-- 2.1 智能体管理
GET /api/agent/list
POST /api/agent/create
PUT /api/agent/{id}/update
DELETE /api/agent/{id}/delete
POST /api/agent/{id}/test -- 测试运行

GET /api/agent/sub-agents
POST /api/agent/sub-agent/create

GET /api/agent/skills
POST /api/agent/skill/create

GET /api/agent/mcp-configs
POST /api/agent/mcp-config/create
POST /api/agent/mcp-config/{id}/test -- 测试 MCP 连接

-- 2.2 流水线配置
GET /api/pipeline-config/stages
PUT /api/pipeline-config/stage/{stageNumber}/update

GET /api/pipeline-config/model-routing
PUT /api/pipeline-config/model-routing/update

-- 2.3 业务知识管理
GET /api/knowledge/rules -- 业务规则列表
POST /api/knowledge/rule/create
PUT /api/knowledge/rule/{id}/update

GET /api/knowledge/domain -- 领域知识列表
GET /api/knowledge/domain/{id}/detail

GET /api/sandbox-config -- 沙箱配置
PUT /api/sandbox-config/update

GET /api/evals/golden-set -- 评测基准
POST /api/evals/run -- 执行评测
GET /api/evals/history -- 评测历史

-- 2.4 租户定制 (tenant_admin 可用)
GET /api/tenant/industry -- 获取本租户行业知识
PUT /api/tenant/industry/update -- 更新行业知识

GET /api/tenant/glossary -- 获取本租户术语表
POST /api/tenant/glossary/create -- 新增术语
PUT /api/tenant/glossary/{id}/update
DELETE /api/tenant/glossary/{id}/delete
```

5.4 自博弈训练引擎接口 (仅 founder)

```
-- 4.1 引擎总控
GET    /api/foundry/engine/status
POST   /api/foundry/engine/start
POST   /api/foundry/engine/stop
PUT    /api/foundry/engine/config    -- 修改参数

-- 4.2 对抗角色
GET    /api/foundry/agent/{role}    -- role: attacker/builder/judge/distiller
PUT    /api/foundry/agent/{role}/update

-- 4.3 训练运行
GET    /api/foundry/dashboard        -- 仪表盘数据
GET    /api/foundry/causal-replay/list
GET    /api/foundry/sandbox-cluster/status

-- 4.4 领域知识进化
GET    /api/foundry/knowledge/patterns
GET    /api/foundry/knowledge/nodes
GET    /api/foundry/knowledge/graph    -- 图谱数据 (D3/ECharts)
GET    /api/foundry/knowledge/stats
GET    /api/foundry/knowledge/versions/{nodeId}
GET    /api/foundry/knowledge/anti-patterns
GET    /api/foundry/knowledge/narratives/{patternId}
GET    /api/foundry/knowledge/cold-start
PUT    /api/foundry/knowledge/forgetting/config

-- 4.5 知识补丁
POST   /api/foundry/patch/create    -- 打包 Patch
GET    /api/foundry/patch/list
POST   /api/foundry/patch/{id}/approve -- 审核通过
POST   /api/foundry/patch/{id}/reject -- 驳回
POST   /api/foundry/patch/{id}/sign  -- 签发
GET    /api/foundry/patch/logs      -- 接收日志
```

六、前端路由与组件

6.1 路由结构 (Vue Router)

```
// router/modules/studio.ts

const studioRoutes = [
  // 一、AI 原生开发平台
  {
    path: '/studio/ai',
    component: () => import('@/layout/studio.vue'),
    meta: { title: 'AI 原生开发平台', icon: 'ai' },
    children: [
      {
        path: 'submit-requirement',
        name: 'AiSubmitRequirement',
        component: () => import('@/views/studio/ai/submit-requirement.vue'),
        meta: { title: '提交需求', roles: 'all' }
      },
    ],
  },
]
```

```

    {
      path: 'generated-systems',
      name: 'AiGeneratedSystems',
      component: () => import('@/views/studio/ai/generated-systems.vue'),
      meta: { title: '已生成系统', roles: 'all', badge: true }
    },
    {
      path: 'ui-templates',
      name: 'AiUiTemplates',
      component: () => import('@/views/studio/ai/ui-templates.vue'),
      meta: { title: 'UI 模板库', roles: 'all' }
    },
    {
      path: 'usage-billing',
      name: 'AiUsageBilling',
      component: () => import('@/views/studio/ai/usage-billing.vue'),
      meta: { title: '用量与计费', roles: 'all', dataScope: 'OWN' }
    }
  ]
},

// 二、智能体与流水线配置
{
  path: '/studio/agent',
  component: () => import('@/layout/studio.vue'),
  meta: { title: '智能体与流水线配置', icon: 'config', roles: ['platform_admin', 'tenant_admin'] },
  children: [
    // 2.1 智能体管理
    {
      path: 'create',
      name: 'AgentCreate',
      component: () => import('@/views/studio/agent/create.vue'),
      meta: { title: '智能体创建与配置', roles: ['platform_admin'], expandPhase: 'A' }
    },
    {
      path: 'sub-agents',
      name: 'SubAgents',
      component: () => import('@/views/studio/agent/sub-agents.vue'),
      meta: { title: '子智能体管理', roles: ['platform_admin'], expandPhase: 'A' }
    },
    {
      path: 'skills',
      name: 'AgentSkills',
      component: () => import('@/views/studio/agent/skills.vue'),
      meta: { title: 'Skills 管理', roles: ['platform_admin'], expandPhase: 'A' }
    },
    {
      path: 'mcp',
      name: 'AgentMcp',
      component: () => import('@/views/studio/agent/mcp.vue'),
      meta: { title: 'MCP 配置', roles: ['platform_admin'], expandPhase: 'A' }
    },
    // 2.2 流水线配置
    {
      path: 'pipeline/stages',
      name: 'PipelineStages',

```

```

    component: () => import('@/views/studio/pipeline/stages.vue'),
    meta: { title: '流水线阶段设置', roles: ['platform_admin'], expandPhase: 'A' }
  },
  {
    path: 'pipeline/model-routing',
    name: 'ModelRouting',
    component: () => import('@/views/studio/pipeline/model-routing.vue'),
    meta: { title: '模型路由策略', roles: ['platform_admin'], expandPhase: 'A' }
  },
  // 2.3 业务知识管理
  {
    path: 'knowledge/rule-editor',
    name: 'RuleEditor',
    component: () => import('@/views/studio/knowledge/rule-editor.vue'),
    meta: { title: '业务规则配置中心', roles: ['platform_admin', 'tenant_admin'],
expandPhase: 'C' }
  },
  {
    path: 'knowledge/domain-knowledge',
    name: 'DomainKnowledge',
    component: () => import('@/views/studio/knowledge/domain-knowledge.vue'),
    meta: { title: '领域知识管理', roles: ['platform_admin', 'tenant_admin'],
expandPhase: 'C' }
  },
  {
    path: 'knowledge/sandbox-config',
    name: 'SandboxConfig',
    component: () => import('@/views/studio/knowledge/sandbox-config.vue'),
    meta: { title: '沙箱部署设置', roles: ['platform_admin'], expandPhase: 'A' }
  },
  {
    path: 'knowledge/evals',
    name: 'EvalsManagement',
    component: () => import('@/views/studio/knowledge/evals.vue'),
    meta: { title: '评测基准管理', roles: ['platform_admin'], expandPhase: 'A' }
  },
  // 2.4 租户定制
  {
    path: 'tenant/industry-knowledge',
    name: 'IndustryKnowledge',
    component: () => import('@/views/studio/tenant/industry-knowledge.vue'),
    meta: { title: '行业知识设置', roles: ['tenant_admin'], expandPhase: 'C' }
  },
  {
    path: 'tenant/glossary',
    name: 'Glossary',
    component: () => import('@/views/studio/tenant/glossary.vue'),
    meta: { title: '业务术语表', roles: ['tenant_admin'], expandPhase: 'C' }
  }
]
},
// 三、JNPF 开发工具箱
{
  path: '/studio/jnpf',
  component: () => import('@/layout/studio.vue'),
  meta: { title: 'JNPF 开发工具箱', icon: 'tools', roles: ['developer'] },
  children: [

```

```

    { path: 'domain-canvas', name: 'DomainCanvas', component: () =>
import('@/views/studio/jnpf/domain-canvas.vue'), meta: { title: '领域模型画板' } } },
    { path: 'arch-designer', name: 'ArchDesigner', component: () =>
import('@/views/studio/jnpf/arch-designer.vue'), meta: { title: '架构图设计器' } } },
    { path: 'decision-table', name: 'DecisionTable', component: () =>
import('@/views/studio/jnpf/decision-table.vue'), meta: { title: '决策表编辑器' } } },
    { path: 'form-designer', name: 'FormDesigner', component: () =>
import('@/views/studio/jnpf/form-designer.vue'), meta: { title: '表单设计器' } } },
    { path: 'dashboard-designer', name: 'DashboardDesigner', component: () =>
import('@/views/studio/jnpf/dashboard-designer.vue'), meta: { title: '大屏设计器' } } },
    { path: 'workflow-designer', name: 'WorkflowDesigner', component: () =>
import('@/views/studio/jnpf/workflow-designer.vue'), meta: { title: '工作流设计器' } } }
  ]
},

```

// 四、自博弈训练引擎

```

{
  path: '/studio/foundry',
  component: () => import('@/layout/studio.vue'),
  meta: { title: '自博弈训练引擎', icon: 'foundry', roles: ['founder'],
requiresTotp: true },
  children: [
    { path: 'engine-control', name: 'EngineControl', component: () =>
import('@/views/studio/foundry/engine-control.vue'), meta: { title: '引擎开关与参数' } } },
    { path: 'agents/attacker', name: 'AgentAttacker', component: () =>
import('@/views/studio/foundry/agents/attacker.vue'), meta: { title: '需求攻击者' } } },
    { path: 'agents/builder', name: 'AgentBuilder', component: () =>
import('@/views/studio/foundry/agents/builder.vue'), meta: { title: '系统构建者' } } },
    { path: 'agents/judge', name: 'AgentJudge', component: () =>
import('@/views/studio/foundry/agents/judge.vue'), meta: { title: '对抗性判官' } } },
    { path: 'agents/distiller', name: 'AgentDistiller', component: () =>
import('@/views/studio/foundry/agents/distiller.vue'), meta: { title: '知识蒸馏师' } } },
    { path: 'dashboard', name: 'FoundryDashboard', component: () =>
import('@/views/studio/foundry/dashboard.vue'), meta: { title: '自博弈仪表盘' } } },
    { path: 'causal-replay', name: 'CausalReplay', component: () =>
import('@/views/studio/foundry/causal-replay.vue'), meta: { title: '因果回放池' } } },
    { path: 'sandbox-cluster', name: 'SandboxCluster', component: () =>
import('@/views/studio/foundry/sandbox-cluster.vue'), meta: { title: '沙箱集群管理' } } },
    { path: 'knowledge/patterns', name: 'KnowledgePatterns', component: () =>
import('@/views/studio/foundry/knowledge/patterns.vue'), meta: { title: '领域模式' } } },
    { path: 'knowledge/nodes', name: 'KnowledgeNodes', component: () =>
import('@/views/studio/foundry/knowledge/nodes.vue'), meta: { title: '知识节点' } } },
    { path: 'knowledge/graph', name: 'KnowledgeGraph', component: () =>
import('@/views/studio/foundry/knowledge/graph.vue'), meta: { title: '知识图谱' } } },
    { path: 'knowledge/stats', name: 'KnowledgeStats', component: () =>
import('@/views/studio/foundry/knowledge/stats.vue'), meta: { title: '使用统计' } } },
    { path: 'knowledge/versions', name: 'KnowledgeVersions', component: () =>
import('@/views/studio/foundry/knowledge/versions.vue'), meta: { title: '版本历史' } } },
    { path: 'knowledge/anti-patterns', name: 'AntiPatterns', component: () =>
import('@/views/studio/foundry/knowledge/anti-patterns.vue'), meta: { title: '反模式记录' } } },

```

```

    { path: 'knowledge/narratives', name: 'Narratives', component: () =>
import('@/views/studio/foundry/knowledge/narratives.vue'), meta: { title: '叙事式说明'
} }},
    { path: 'knowledge/cold-start', name: 'ColdStart', component: () =>
import('@/views/studio/foundry/knowledge/cold-start.vue'), meta: { title: '冷启动种子'
} }},
    { path: 'knowledge/forgetting', name: 'Forgetting', component: () =>
import('@/views/studio/foundry/knowledge/forgetting.vue'), meta: { title: '遗忘机制' }
},
    { path: 'patch/review', name: 'PatchReview', component: () =>
import('@/views/studio/foundry/patch/review.vue'), meta: { title: 'Patch 审核与签发' }
},
    { path: 'patch/logs', name: 'PatchLogs', component: () =>
import('@/views/studio/foundry/patch/logs.vue'), meta: { title: 'Patch 接收日志' } }
  ]
}
]

```

6.2 权限拦截器

// permission.ts - 路由守卫

```

router.beforeEach(async (to, from, next) => {
  const userStore = useUserStore()

  // 1. 检查登录
  if (!userStore.token) return next('/login')

  // 2. 检查角色
  const requiredRoles = to.meta.roles
  if (requiredRoles && requiredRoles !== 'all') {
    const hasRole = requiredRoles.some(role => userStore.roles.includes(role))
    if (!hasRole) return next('/403')
  }

  // 3. TOTP 门禁（自博弈训练引擎）
  if (to.meta.requiresTotp && !userStore.totpverified) {
    return next('/studio/foundry/totp-verify')
  }

  // 4. 数据范围（后端过滤，前端只需传参数）
  // dataScope 通过 API 请求头自动注入
  next()
})

```

6.3 核心组件清单

```

src/views/studio/
├─ ai/
│   └─ submit-requirement.vue      ← 五阶段流水线主界面
│       └─ components/
│           └─ AiChatPanel.vue      ← 富媒体对话面板(SSE流式)
│           └─ IrDiffViewer.vue     ← IR 差异查看器
│           └─ StageProgress.vue    ← 五阶段进度条
│           └─ StageConfirmDialog.vue ← 阶段确认对话框
│       └─ generated-systems.vue    ← 已生成系统列表

```

└─ components/	
└─ SystemCard.vue	← 系统卡片(含红点/沙箱链接/下载)
└─ DeployGuide.vue	← 部署说明展示
└─ ui-templates.vue	← 模板库(含Tab切换)
└─ components/	
└─ TemplateMarket.vue	← Tab1: 模板市场
└─ Templateworkshop.vue	← Tab2: 模板工坊
└─ usage-billing.vue	← 用量与计费(含Tab切换)
└─ components/	
└─ TokenSummary.vue	← Token 用量汇总
└─ CallLogTable.vue	← AI 调用明细表
└─ BillingPanel.vue	← 续费管理
└─ agent/	
└─ create.vue	← 智能体创建与配置
└─ sub-agents.vue	← 子智能体管理
└─ skills.vue	← Skills 管理
└─ mcp.vue	← MCP 配置
└─ pipeline/	
└─ stages.vue	← 流水线阶段设置
└─ model-routing.vue	← 模型路由策略
└─ knowledge/	
└─ rule-editor.vue	← 业务规则配置中心
└─ domain-knowledge.vue	← 领域知识管理
└─ sandbox-config.vue	← 沙箱部署设置
└─ evals.vue	← 评测基准管理
└─ tenant/	
└─ industry-knowledge.vue	← 行业知识设置
└─ glossary.vue	← 业务术语表
└─ jnpf/	
└─ domain-canvas.vue	← 领域模型画板
└─ arch-designer.vue	← 架构图设计器
└─ decision-table.vue	← 决策表编辑器
└─ form-designer.vue	← 表单设计器(已有)
└─ dashboard-designer.vue	← 大屏设计器(已有)
└─ workflow-designer.vue	← 工作流设计器
└─ foundry/	
└─ engine-control.vue	← 引擎开关与参数
└─ agents/	
└─ attacker.vue	← 需求攻击者配置
└─ builder.vue	← 系统构建者配置
└─ judge.vue	← 对抗性判官配置
└─ distiller.vue	← 知识蒸馏师配置
└─ dashboard.vue	← 自博弈仪表盘
└─ causal-replay.vue	← 因果回放池
└─ sandbox-cluster.vue	← 沙箱集群管理
└─ knowledge/	
└─ patterns.vue	← 领域模式
└─ nodes.vue	← 知识节点
└─ graph.vue	← 知识图谱(D3/Echarts)
└─ stats.vue	← 使用统计

	— versions.vue	← 版本历史
	— anti-patterns.vue	← 反模式记录
	— narratives.vue	← 叙事式说明
	— cold-start.vue	← 冷启动种子
	— forgetting.vue	← 遗忘机制
— patch/		
	— review.vue	← Patch 审核与签发
	— logs.vue	← Patch 接收日志

组件总计：46 个 Vue 文件

七、Sprint 执行计划

Sprint 1：权限基础设施（2 周）

目标：权限架构一次建好，后续所有 Sprint 在此之上开发。

天	任务	产出
D1-2	后端：建表 BASE_MENU + BASE_USER_ROLE + 初始数据 SQL	数据库就绪
D3	后端：MenuService — 获取用户菜单树 + 角色过滤逻辑	/api/menu/user-menus 可调用
D4	后端：权限拦截中间件 — 读取路由 meta.roles 做服务端二次校验	403/401 矩阵测试通过
D5	后端：TenantMiddleware — TenantId 全链路注入	多租户数据隔离
D6-7	前端：路由守卫 + 动态菜单渲染（从 API 拉菜单树，按角色渲染侧边栏）	侧边栏按角色正确显示/隐藏
D8	前端：TOTP 二次认证页面 + FounderGuard 拦截	founder 菜单需认证
D9	前端：红点提示组件 + BASE_MENU_BADGE 读写	"已生成系统"冒红点
D10	联调 + 权限矩阵测试（6 个角色 × 4 个一级菜单 = 24 组验证）	权限矩阵全绿

验收标准：

- 6 个角色登录后看到的菜单各不相同
- 非 founder 看不到自博弈训练引擎
- founder 未通过 TOTP 时只能看到认证页面
- tenant_admin 看不到"智能体创建与配置"，但能看到"行业知识设置"
- developer 看到 JNPF 开发工具箱，business_expert 看不到
- 红点机制可用

Sprint 2：AI 原生开发平台核心（2 周）

目标：提交需求 → 五阶段流水线 → 已生成系统，跑通核心主链路。

天	任务	产出
D1-2	后端：建表 BASE_AI_PIPELINE + BASE_AI_PIPELINE_MESSAGE + BASE_AI_GENERATED_PROJECT	数据库就绪
D3	后端：PipelineService.CreateSession — 创建流水线会话	API 可调
D4-5	后端：接入 DeepSeek Gateway (F-9.1) + 需求分析师 Agent (F-9.4) + 阶段 1 逻辑	AI 能对话并追问

天	任务	产出
D6	前端: submit-requirement.vue 主框架 + StageProgress 五阶段进度条	页面骨架
D7-8	前端: AiChatPanel (SSE 流式) + 阶段确认对话框	能和 AI 对话
D9	后端: 生成系统后写入 BASE_AI_GENERATED_PROJECT + 更新 BASE_MENU_BADGE	红点机制联动
D10	前端: generated-systems.vue 系统卡片 + 沙箱链接 + 源码下载 + 部署文档	已生成系统页面

验收标准:

- 用户描述需求 → AI 追问至少 3 个问题 → 用户确认 → 生成需求文档
- 已生成系统页面显示系统列表, 带沙箱链接 (admin/123456)
- 源代码 ZIP 和部署文档可下载
- 系统生成后, 侧边栏"已生成系统"冒红点带数字

Sprint 3: 智能体配置 A 部分 (2 周)

目标: 平台管理员能配置智能体、Prompt 模板、子智能体、Skills、MCP。

天	任务
D1-2	建表 BASE_AI_PROMPT_TEMPLATE + 智能体配置表 + Skills 表 + MCP 配置表
D3-4	AgentService CRUD + Prompt 模板插值引擎 + 智能体测试运行
D5	SubAgentService + OrchestratorAgent 调度配置
D6	SkillsService + McpService
D7-8	前端: 四个页面 (create / sub-agents / skills / mcp)
D9	前端: 智能体测试运行面板 (输入测试用例 → 查看 AI 响应)
D10	联调: 创建一个完整智能体 → 关联 Skills 和 MCP → 测试通过

验收标准:

- 平台管理员可创建智能体, 配置 Prompt、关联子智能体/Skills/MCP
- 测试运行面板可即时测试智能体响应
- 只有 platform_admin 能看到这些菜单

Sprint 4: 流水线配置 + 知识管理 A 部分 (2 周)

天	任务
D1-2	流水线阶段配置 Service + 模型路由配置 Service
D3	评测基准 golden set 管理 + eval-runner 执行引擎
D4	沙箱部署配置 Service (资源限制、并发数)
D5	前端: 流水线阶段设置页面 (五阶段各自的行为/模板/确认门槛)
D6	前端: 模型路由策略页面
D7	前端: 评测基准管理页面
D8	前端: 沙箱部署设置页面
D9	集成: 流水线运行时读取配置 (阶段设置 + 模型路由 + 沙箱参数)
D10	端到端测试: 修改配置 → 运行流水线 → 验证配置生效

Sprint 5: C 部分扩展 + 业务规则 (1 周)

天 任务

- D1 建表 BASE_TENANT_INDUSTRY + BASE_TENANT_GLOSSARY
 - D2 TenantConfigService (行业知识 CRUD + 术语表 CRUD + 自动拼接 SystemPrompt)
 - D3 前端: 行业知识设置页面 + 业务术语表页面
 - D4 前端: 业务规则配置中心开放 tenant_admin 权限 (编辑本租户)
 - D5 前端: 领域知识管理开放 tenant_admin 权限 (只读)
- 集成 验证: 租户 A 设置行业知识 → 提交需求 → AI 上下文包含行业描述

验收标准:

- tenant_admin 登录后能看到"租户定制"菜单下的两个子菜单
- 设置行业描述后, AI 对话中会用行业术语
- tenant_admin 看不到 platform_admin 专属菜单

Sprint 6: UI 模板库 + 用量计费 (1 周)

天 任务

- D1 建表 UI 模板表 + 接入 BASE_AI_CALL_LOG 查询接口
- D2 前端: UI 模板库页面 (模板市场 Tab + 模板工坊 Tab)
- D3 前端: Token 用量统计页面 (按供应商/按阶段/按用户 聚合图表)
- D4 前端: AI 调用明细列表 (分页、筛选、导出)
- D5 前端: 订阅续费管理页面

总时间线

sprint 1		权限基础设施	2周
sprint 2		AI 原生开发核心	2周
sprint 3		智能体配置 A	2周
sprint 4		流水线配置+知识A	2周
sprint 5		C 部分扩展	1周
sprint 6		模板库+用量计费	1周
合计			10周

八、扩展到 B 阶段的前置条件

当决定启动 B 阶段 (租户深度定制智能体) 时, 检查以下前置条件:

条件

- A+C 所有 Sprint 验收通过
- 至少 5 个真实租户在使用
- 至少 3 个租户提出"想定制 AI 行为"的需求
- Prompt 注入防护机制就绪
- 租户版智能体的资源隔离方案确定 (Token 上限、调用频次)

验证方式

- 权限矩阵全绿
- 租户数统计
- 客户反馈记录
- 安全测试报告
- 架构文档

满足后，B 阶段改动清单：

- 1. BASE_MENU.F_RequiredRoles 加入 tenant_admin
- 2. 前端根据 F_TenantViewConfig 渲染简化版表单
- 3. 新增"我的智能体"页面（从模板复制→改配置→测试→启用）
- 4. 后端增加租户智能体的数据隔离（F_TenantId 全链路）

预计工作量：2 周。

附录 A：文件索引

文件	路径
菜单	
表	backend/sql/v5.2_001_menu.sql
DDL	
权限	
中间件	backend/application/JNPF.API.Entry/Middleware/MenuPermissionMiddleware.cs
TOTP 认证	backend/application/JNPF.API.Entry/Middleware/FounderGuardMiddleware.cs
菜单服务	backend/application/JNPF.Application/Studio/MenuService.cs
流水线服务	backend/application/JNPF.Application/Studio/PipelineService.cs
智能体服务	backend/application/JNPF.Application/Studio/AgentService.cs
路由守卫	jnpf-web-vue3/src/permission.ts
动态菜单	jnpf-web-vue3/src/layout/components/Sidebar.vue
IR 真源	jnpf-web-vue3/src/core/ir/types.ts

附录 B：文档版本

版本	日期	变更
v1.0	2026-06-17	初始版本，覆盖 A+C 全部设计 B 阶段仅预留字段，不含实施设计